

THE **AI** CODEBASE ANALYST

FROM ZERO TO A WORKING AGENT

LESSON 1
WORKSPACE & OBSERVABILITY

Lesson 1: The Professional Workspace & Your First Observable Call

"You can't manage what you can't measure."

— Peter Drucker

The "Why": The Ghost in the Machine

You've seen the magic. You sent a prompt to an LLM and watched, astonished, as it produced something coherent, creative, or uncannily accurate.

But you are also here because you've felt that magic flicker. The initial excitement of a "Hello, World!" chatbot gives way to the sobering reality of a real-world project. You've encountered the ghost in the machine: that moment when your application gives a bizarre output, and you have absolutely no idea why.

This is the chasm between a prototype and a production system. To bridge it, we must start with a brutally honest diagnosis:

Every AI Agent is a Liar, a Slacker, and a Thief.

- It is a **Liar**: It hallucinates, confidently inventing facts when it doesn't know the answer.
- It is a **Slacker**: It will take the easiest path, often giving a superficial answer when a deeper one is required.
- It is a **Thief**: It can inadvertently plagiarize or reveal sensitive data it was trained on.

Our job as professional AI engineers is not to "fix" these flaws—they are inherent to the technology. Our job is to build a **system of accountability** around our brilliant but untrustworthy new hire. This lesson is about building the secure, observable workspace where that accountability begins.

The "What": The Four Pillars of a Professional Start

Before writing a single line of agentic logic, we will build a foundation that answers four critical questions:

1. **Security:** How do I manage secret API keys without hardcoding them and leaking them on GitHub?
2. **Flexibility:** How do I avoid locking my application into a single AI model provider?
3. **Cost:** How can I control expenses by choosing the right model for the right task?
4. **Observability:** How can I see *exactly* what happened inside my application?

We will solve these foundational problems by establishing a professional project structure from day one, built on four industry-standard patterns:

1. **Isolated Development Sanctuaries** (Virtual Environments) to keep our project's dependencies clean and reproducible.
2. **Secure Credential Management** (A .env file) to store secret API keys safely outside our source code.
3. **The Universal Power Adapter** (OpenRouter) as a unified gateway to access dozens of LLMs, giving us maximum flexibility and cost control.
4. **The Flight Recorder** (LangSmith) to trace our application's execution from the very first API call, giving us a "glass box" view into our system.

The "How" (The Build): Step-by-Step

Let's move from theory to terminal. We will create our project, acquire our keys, and install the necessary libraries.

1. Create Your Project Directory & Virtual Environment

Open your terminal and run these commands:

```
mkdir codebase-analyst
```

```
cd codebase-analyst
```

```
# Create the virtual environment
```

```
# For macOS/Linux
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
# For Windows (PowerShell or Command Prompt)
```

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

Your terminal prompt should now be prefixed with (venv). You are now working inside your isolated sanctuary.

2. Acquire Your API Keys

This is the only lesson that requires external account setup. Taking five minutes to do this now will ensure a smooth experience for the rest of the course.

- **OpenRouter Key:** Go to [OpenRouter.ai](https://openrouter.ai), sign up, and create a new API key.
- **LangSmith Key:** Go to [LangSmith](https://smith.langchain.com), sign up, and create a new API key within your default project.

3. Create the .env File for Secrets

Inside your codebase-analyst folder, create a new file named .env. This file will hold your secret keys.

SECURITY WARNING: Never Commit Your .env File

This file contains your secret keys. If you use Git for version control (which you should),

ensure you add `.env` to your `.gitignore` file immediately. Exposing this key is a significant security risk.

! A Note on Project Naming: When you create a key, the LangSmith UI will often suggest a default project name like `pr-respectful-rail-4` in its setup instructions. We will **intentionally override this**. A core principle of production engineering is deliberate naming for clarity and organization. We will set our own project name in the `.env` file.

Add your keys and configuration. If a project with the name you specify doesn't exist, LangSmith will create it for you on the first run.

```
modern LANGSMITH_* variables for clarity and explicit control.
# .env - This file should NOT be committed to Git.

# --- Model Provider Configuration ---
# Key for model access via OpenRouter, our flexible gateway.
OPENROUTER_API_KEY="sk-or-..."
OPENROUTER_BASE_URL="https://openrouter.ai/api/v1"

# --- Observability Configuration (LangSmith) ---
# Tells LangChain libraries to send traces to LangSmith.
LANGSMITH_TRACING="true"
# The API endpoint for the LangSmith service.
LANGSMITH_ENDPOINT="https://api.smith.langchain.com"
# Your specific API key for authenticating with LangSmith.
LANGSMITH_API_KEY="ls_..."
# (Optional but Recommended) Specifies the project name in
# LangSmith.
# This keeps your runs organized. If not set, it defaults to
# "default".
LANGSMITH_PROJECT="codebase-analyst-agent"
```

4. Install Dependencies

We'll install the necessary libraries from the LangChain ecosystem to interact with LLMs and enable tracing.

```
pip install python-dotenv langchain langchain-openai langsmith
```

Your professional workspace is now ready.

Code Walkthrough: The First Observable Call

Now, let's write the Python script that makes our first, fully observable API call.

- **Filename:** 01_first_observable_call.py

```
# 01_first_observable_call.py

import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

# --- 1. Load Environment Variables (The Cornerstone of Security)
---
# This must be the first step to ensure all required keys are loaded
before use.
load_dotenv()

# --- 2. Initialize the Language Model Client (The Universal
Adapter) ---
# We initialize our primary client for interacting with LLMs.
llm = ChatOpenAI(
    # We point the client to the OpenRouter gateway.
    base_url="https://openrouter.ai/api/v1",
```

```
# The API key is loaded securely from the environment.
api_key=os.getenv("OPENROUTER_API_KEY"),
# We explicitly select our professional workhorse model.
model="google/gemini-2.5-flash",
)

# --- 3. The LLM Invocation (The Core Logic) ---
# This is the core logic where we interact with the LLM.
def get_llm_summary(text_to_summarize: str) -> str:
    """Invokes the LLM to summarize a given text."""

    print("Requesting summary from our Gemini model via
    OpenRouter...")
    try:
        # The .invoke() method sends the request to the LLM.
        response = llm.invoke(text_to_summarize)
        # The summary text is in the `content` attribute of the
        response.
        return response.content
    except Exception as e:
        print(f"An error occurred: {e}")
        return None

# --- 4. Run the Script (The Test Case) ---
if __name__ == "__main__":
    # The abstract from the famous "Attention Is All You Need"
    paper.
    transformer_abstract = """
    The dominant sequence transduction models are based on complex
    recurrent or
    convolutional neural networks, including an encoder and a
```

```
decoder. The best
    performing models also connect the encoder and decoder through
an attention
    mechanism. We propose a new simple network architecture, the
Transformer,
    based solely on attention mechanisms, dispensing with recurrence
and convolutions
    entirely. Experiments on two machine translation tasks show
these models to
    be superior in quality while being more parallelizable and
requiring significantly
    less time to train.
    """
```

```
ai_summary = get_llm_summary(transformer_abstract)

if ai_summary:
    print("\n--- AI-Generated Summary (Gemini 2.5 Flash) ---")
    print(ai_summary)
```

Deconstructing the Script

This script is simple, but it's built on several powerful professional patterns.

- **Section 1: load_dotenv():** This function is our security foundation. It reads your .env file and loads the variables into the script's environment, making them accessible via os.getenv() without ever exposing them in the code.
- **Section 2: The ChatOpenAI Client:** This is where two pieces of "magic" happen:
 - **Observability (Convention over Configuration):** The client is pre-programmed to look for the LANGSMITH_TRACING,

LANGSMITH_ENDPOINT, and LANGSMITH_API_KEY environment variables. Because we set them in our .env file, it automatically configures itself to send detailed trace data to our specific project in LangSmith. We get a "glass box" view into our system with zero lines of code.

- **Flexibility (The OpenAI-Compatible Standard):** We're using a client named ChatOpenAI to talk to OpenRouter and a Google model. This works because the OpenAI API has become the industry's "USB-C" standard. Services like OpenRouter are designed with a compatible port, giving us incredible portability. The same code could be pointed to a different service tomorrow with only a one-line change to the base_url.
- **Section 3: .invoke() and .content:** The raw response from an LLM is a complex JSON object. The LangChain client simplifies this. .invoke() handles the complex HTTP request formatting, and .content provides easy access to the AI's actual text message.
- **Section 4: if __name__ == "__main__":** This is a standard Python idiom that makes our code reusable. It creates a block that only runs when you execute the file directly.

The Integrated Run: Putting It All Together

Prerequisites:

- Python 3.9+ installed and an active (venv).
- Valid API keys for OpenRouter and LangSmith in your .env file.

Run Command:

From your terminal, execute the script:

```
python 01_first_observable_call.py
```

Expected Console Output:

```
Requesting summary from our Gemini model via OpenRouter...
```

```
--- AI-Generated Summary (Gemini 1.5 Flash) ---
```

```
The Transformer, a new network architecture based solely on
attention mechanisms, outperforms traditional recurrent and
convolutional models in machine translation tasks by being superior
in quality, more parallelizable, and requiring significantly less
training time.
```

Analysis: The System-Level View

Your script performed a simple action, but it triggered a sophisticated, observable workflow. Go to your LangSmith project now. You will see a new trace record for this run. Clicking on it will reveal every detail: the exact model used, the input and output text, the latency, and more. This is your "glass box."

The screenshot displays the LangSmith interface for a project named 'codebase-analyst-agent'. The top navigation bar includes 'Personal', 'Tracing Projects', and 'codebase-analyst-agent'. Below this, there are tabs for 'Runs', 'Threads', 'Evaluators', and 'Automations'. The 'Runs' tab is active, showing a table of trace records. The table has columns for 'Name', 'Input', 'Output', 'Error', and 'Start Time'. A single record is visible with the name 'ChatOpenAI', input 'human: The dominant sequence transd...', output 'ai: This passage i...', and start time '10/10/2025, 4:53:59.'. To the right of the table, there is a 'Stats' panel showing 'Run Count' as 1, 'Total Tokens' as 394 / \$0.00, and 'Error Rate' as 0%. Below the stats, there is a 'Filter Shortcuts' section with a search input and several filter options.

Name	Input	Output	Error	Start Time
ChatOpenAI	human: The dominant sequence transd...	ai: This passage i...		10/10/2025, 4:53:59.

Stats

- Run Count: 1
- Total Tokens: 394 / \$0.00
- Error Rate: 0%

Filter Shortcuts

Input

- Search input...
- messages.id
- messages.id == "huma...
- messages.id == "langc...
- messages.id == "mess...
- messages.id == "scha

Figure 1-1: The "glass box" in action. Your first successful trace in LangSmith, capturing the exact input, output, and metadata of your API call.

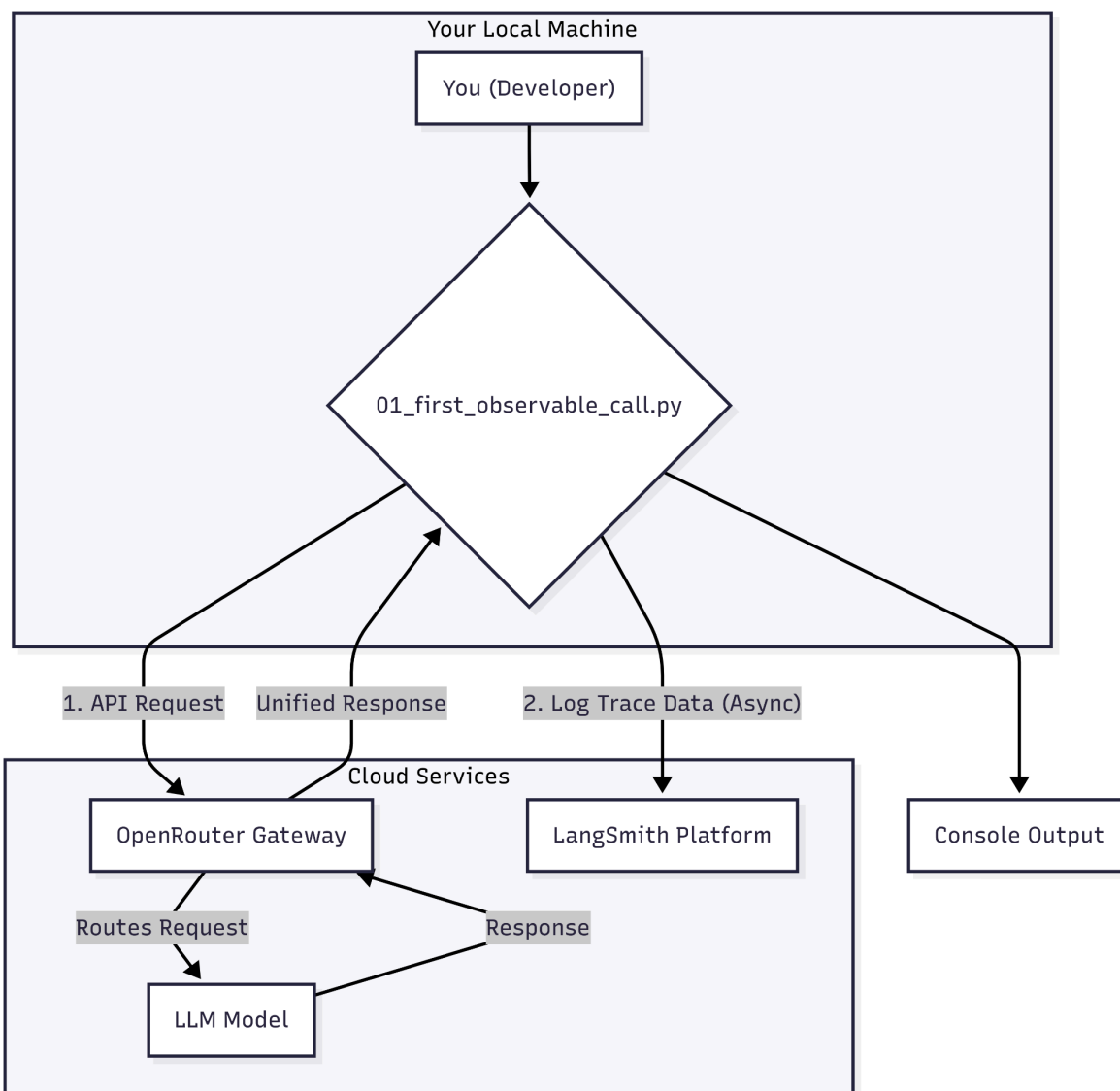


Figure 1-2 The Observable API Gateway Pattern. Your code makes one call that is simultaneously routed to the model provider and traced by the observability platform.

💡 Pro Tip: Choosing Your Workhorse Model

Choosing a model is a key engineering decision. We selected google/gemini-flash-2.5 as our primary model for this course for strategic reasons:

- **Low Cost & High Speed:** "Flash" models are optimized for rapid, inexpensive responses, which is ideal for development and for many production tasks.

- **Massive Context Window:** It has a huge context window, which gives us more flexibility for complex RAG tasks later.
 - **Excellent Tool-Using Capabilities:** Its ability to use tools is a critical feature for building powerful agents. By starting with it now, we are building on a strong foundation.
-

Summary & Key Takeaways

Congratulations! You have successfully built a secure, flexible, and fully observable foundation for your AI agent. This professional starting point is the most critical step for any serious AI project.

Mental Models You've Gained:

- **The Professional Stack:** You know that production-grade AI starts with secure keys (.env), clean environments (venv), a flexible model gateway (OpenRouter), and day-one observability (LangSmith).
- **The Gateway Principle:** You can decouple your code from a specific model provider, gaining massive architectural flexibility. This is a form of "entropy arbitrage"—managing the chaos of the model market by creating a stable interface.
- **The Glass Box Principle:** You understand that you don't have to guess what your application is doing. With LangSmith, you can see every step, input, and output, turning a black box into a transparent system to manage your "liar, slacker, and thief."

What's Next

We've made a single, observable call. We proved our setup works. But this is not an agent. An agent can reason, plan, and take multiple steps. Our script is a simple, one-shot tool.

In Lesson 2, we will build on this solid foundation to construct the "brain" of a true agent using LangGraph. We will create our agent as a **State Machine**—the most important pattern in agentic AI—giving it the ability to reason, remember, and setting the stage for it to solve much more complex problems.

P.S. Have questions about setting up your environment or your first LangSmith trace? Your first stop for help is the #👉-foundations-help-feedback [channel on Discord](#).